

GenAI Code Comparison and Final Program

EECS 348 Assignment 2 | Samyu Aitha | September 16, 2026

Summary

I compared programs from Microsoft Copilot and Anthropic Claude. Both programs correctly model the priority rules with a hand-written list-based MaxHeap, but both generated Python instead of the required C. I selected Claude's program as the conceptual baseline because it accepts the grader's test-file path from the command line, matches the required blank-line formatting more closely, handles empty READ and NEXT commands, and separates heap logic from email logic more clearly. I then translated and strengthened that design as a C11 program and tested it against the sample file and additional edge cases.

1 Programs and Access

Microsoft Copilot

I accessed Microsoft Copilot through Copilot's chat interface on my computer and pasted the prompt below into the chat.

Anthropic Claude

I accessed Claude through the Claude website in a web browser and pasted the exact same prompt into the chat.

2 Exact Prompt Used for Both Programs

Generate a complete program for this assignment. The program must prioritize a CEO's emails using a MaxHeap implemented from scratch with a list-based implementation; do not use a pre-existing heap module. Read a test file containing EMAIL <sender category>,<subject>,<MM-DD-YYYY>, NEXT, READ, and COUNT commands. Category priority is Boss, Subordinate, Peer, ImportantPerson, then OtherPerson. Within the same category, the newest date has priority. NEXT displays Sender, Subject, and Date without removing the email; READ removes the highest-priority email without displaying it; COUNT displays the unread count. Handle NEXT or READ on an empty queue, repeated NEXT commands, and repeated READ commands. The grader will provide a different test file. The required implementation language is C.

3 Correctness Analysis

Copilot

The core priority logic is correct: category rank is compared before date, and `datetime.strptime` makes chronological comparison reliable. `push`, `peek`, and `pop` preserve MaxHeap behavior. Empty NEXT and READ do not crash. However, the program is not a valid assignment submission because it is Python rather than C. It also hardcodes `commands.txt` instead of accepting the grader's file path, so the grader would need to rename its file. Its output omits the blank lines shown in the sample. Its `split(',')` is acceptable under the stated rule that subjects contain no commas, but malformed input causes an exception. Equal category and equal date have no explicit tie rule; either equal email can appear first, which does not violate the given requirements.

Claude

Claude also implements the category-first, newest-date-second rule correctly and uses a custom list-based MaxHeap. It correctly accepts exactly one test-file path, keeps NEXT non-destructive, makes READ on an empty heap harmless, and prints the sample's blank lines. Its output for an empty NEXT is clear. The main failure is again that it is Python rather than required C. Parsing also does not trim spaces around comma-separated fields, so an input written as 'Boss, Subject, 01-01-2026' would produce values with leading spaces and could fail the category lookup. Under the exact assignment format, its sample behavior is otherwise correct.

4 Execution Time Analysis

Operation	Copilot	Claude	Reason
EMAIL / insertion	$O(\log n)$	$O(\log n)$	Append once, then move up at most the heap height.
NEXT / peek	$O(1)$	$O(1)$	The maximum-priority email is stored at index 0.
READ / removal	$O(\log n)$	$O(\log n)$	Replace the root and move down at most the heap height.
COUNT	$O(1)$	$O(1)$	Python list length is stored and returned directly.
Whole file	$O(m \log n)$	$O(m \log n)$	Worst case for m commands with at most n unread emails.

The two programs have the same asymptotic running time. Claude's iterative sift-down avoids recursive call overhead, while Copilot's recursive sift-down has at most $O(\log n)$ call depth. This difference is small, but the iterative version is more predictable.

5 Space Complexity Analysis

Both programs store n Email objects in a Python list, so the main data structure uses $O(n)$ space. Copilot parses each date into a datetime object, while Claude stores a three-integer tuple. Both are $O(1)$ extra space per email and therefore $O(n)$ overall. Copilot's recursive sift-down uses $O(\log n)$ temporary call-stack space in the worst case. Claude's iterative sift-up and sift-down use $O(1)$ auxiliary space per operation, giving it a small space advantage. Neither program duplicates the whole heap during insertion or removal.

6 Maintainability Analysis

Copilot

Copilot's code is shorter and easy to follow. `Email.__gt__` keeps the comparison near the data, and the method names are clear. Still, the hardcoded filename makes reuse and grading less convenient. Date comparison depends on importing datetime, and there is little top-level documentation. Output formatting is spread through `process_commands`, and recursive sift-down is slightly less direct to debug.

Claude

Claude's version has a usage comment, command-line validation, index helper methods, a single `priority_key` method, a display method, and clear separation between MaxHeap, Email, command processing, and main. These choices make individual parts easier to test or replace. The main maintainability weakness is that whitespace parsing and input validation are limited. Some divider comments are long, but the structure is consistent and readable.

7 Selection and Justification

I selected Claude's program as my baseline. This was not based only on style. It has four concrete advantages: it takes the test filename as a command-line argument, its sample formatting is closer to the required output, all heap repairs are iterative and use $O(1)$ auxiliary space, and its `priority_key` centralizes the complete comparison rule. Both generated programs still fail the language requirement, so neither could be submitted unchanged. Claude provided the cleaner structure to translate into C and improve.

8 Improvements Made to the Final C Program

Correctness

I rewrote the program in C11, which fixes the largest rubric failure. The program accepts the grader's file path through `argv`, parses the first two commas so spaces remain valid inside subjects, trims optional surrounding whitespace, compares category before YYYYMMDD, and uses arrival order only as a deterministic final tie-breaker. Empty NEXT prints a clear message, empty READ does nothing, repeated NEXT does not remove anything, and repeated READ removes one maximum each time. File-open, allocation, invalid category, invalid date format, and malformed EMAIL errors are handled instead of causing undefined behavior.

Execution time

The final list-based heap keeps EMAIL and READ at $O(\log n)$, NEXT and COUNT at $O(1)$, and total processing at $O(m \log n)$ in the worst case. It uses iterative sift-up and sift-down, so it avoids recursive function-call overhead. The dynamic list doubles when full, making append $O(1)$ amortized before the $O(\log n)$ heap repair.

Space complexity

The final heap uses $O(n)$ space for n unread emails and $O(1)$ auxiliary space for each heap operation. Strings are allocated only to fit their contents. Unlike recursive sift-down, the final version does not require an $O(\log n)$ call stack. Every owned string and the heap list are freed, including emails left unread when the file ends.

Maintainability

I separated category ranking, date conversion, comparison, parsing, heap operations, cleanup, and file processing into small functions. Named structures and fields make the data model clear. The Makefile enables one consistent strict build. The prologue identifies the program, author, dates, inputs, output, revisions, collaborators, and sources. Detailed block and line comments explain both behavior and authorship.

9 Verification

I compiled the final source with `gcc -std=c11 -Wall -Wextra -Werror -pedantic`. It compiled without warnings. I ran the supplied sample commands and compared the complete output to the required sample; they matched. I also checked the hidden-test behaviors described in the instructions: NEXT on empty, READ on empty, two NEXT commands without READ, two READ commands without NEXT, category ordering, newer-date ordering within a category, and spaces in subject lines.

Appendix A Microsoft Copilot Raw Output

The following is the complete code returned by Microsoft Copilot.

```
from datetime import datetime

# Map categories to integer priorities (higher number = higher priority)
CATEGORY_PRIORITY = {
    "Boss": 5,
    "Subordinate": 4,
    "Peer": 3,
    "ImportantPerson": 2,
    "OtherPerson": 1
}

class Email:
    def __init__(self, category, subject, date_str):
        self.category = category
        self.subject = subject
        self.date_str = date_str
        self.priority = CATEGORY_PRIORITY.get(category, 0)
        # Parse date MM-DD-YYYY for direct comparison
        self.date = datetime.strptime(date_str, "%m-%d-%Y")

    def __gt__(self, other):
        # Primary comparison: higher category priority
        if self.priority != other.priority:
            return self.priority > other.priority
        # Secondary comparison: newer date takes priority
        return self.date > other.date

class MaxHeap:
    def __init__(self):
        self.heap = []

    def size(self):
        return len(self.heap)

    def is_empty(self):
        return len(self.heap) == 0

    def push(self, email):
        self.heap.append(email)
        self._sift_up(len(self.heap) - 1)

    def peek(self):
        if self.is_empty():
            return None
        return self.heap[0]

    def pop(self):
        if self.is_empty():
            return None
        if len(self.heap) == 1:
            return self.heap.pop()

        root = self.heap[0]
        self.heap[0] = self.heap.pop()
        self._sift_down(0)
        return root

    def _sift_up(self, index):
        parent = (index - 1) // 2
        while index > 0 and self.heap[index] > self.heap[parent]:
            self.heap[index], self.heap[parent] = self.heap[parent], self.heap[index]
            index = parent
            parent = (index - 1) // 2

    def _sift_down(self, index):
        max_idx = index
        left = 2 * index + 1
        right = 2 * index + 2
        n = len(self.heap)

        if left < n and self.heap[left] > self.heap[max_idx]:
            max_idx = left
        if right < n and self.heap[right] > self.heap[max_idx]:
            max_idx = right

        if index != max_idx:
            self.heap[index], self.heap[max_idx] = self.heap[max_idx], self.heap[index]
            self._sift_down(max_idx)

def process_commands(file_path):
```

```

max_heap = MaxHeap()

with open(file_path, 'r') as file:
    for line in file:
        line = line.strip()
        if not line:
            continue

        if line.startswith("EMAIL "):
            # Extract portion after "EMAIL "
            data = line[6:].strip()
            # Split comma-separated components: category, subject, date
            category, subject, date_str = [item.strip() for item in data.split(',')]
            email = Email(category, subject, date_str)
            max_heap.push(email)

        elif line == "COUNT":
            print(f"There are {max_heap.size()} emails to read.")

        elif line == "NEXT":
            email = max_heap.peek()
            if email:
                print("Next email:")
                print(f"    Sender: {email.category}")
                print(f"    Subject: {email.subject}")
                print(f"    Date: {email.date_str}")
            else:
                print("No emails to display.")

        elif line == "READ":
            max_heap.pop()

# Example Usage
if __name__ == "__main__":
    process_commands("commands.txt")

```

Appendix B Anthropic Claude Raw Output

The following is the complete code returned by Anthropic Claude.

```
"""
CEO Email Priority Queue
-----
Reads a test file of commands (EMAIL, NEXT, READ, COUNT) and processes the
CEO's inbox using a priority queue built on top of a hand-written, list-based
MaxHeap (no built-in heap modules used).

Priority order (highest to lowest):
    Boss > Subordinate > Peer > ImportantPerson > OtherPerson
Within the same sender category, the newest email (by date) has higher
priority than an older one.

Usage:
    python ceo_email_priority.py <test_file>
"""

import sys

# -----
# MaxHeap: list-based implementation built entirely from scratch.
# -----
class MaxHeap:
    def __init__(self):
        self._data = [] # each element is an Email object

    # -- index helpers -----
    @staticmethod
    def _parent(i):
        return (i - 1) // 2

    @staticmethod
    def _left(i):
        return 2 * i + 1

    @staticmethod
    def _right(i):
        return 2 * i + 2

    def _swap(self, i, j):
        self._data[i], self._data[j] = self._data[j], self._data[i]

    # -- comparison: True if item at index a has HIGHER priority than b --
    def _greater(self, a, b):
        return self._data[a].priority_key() > self._data[b].priority_key()

    # -- core heap operations -----
    def push(self, item):
        self._data.append(item)
        self._sift_up(len(self._data) - 1)

    def _sift_up(self, i):
        while i > 0:
            p = self._parent(i)
            if self._greater(i, p):
                self._swap(i, p)
                i = p
            else:
                break

    def pop_max(self):
        """Remove and return the highest-priority item."""
        if not self._data:
            return None
        top = self._data[0]
        last = self._data.pop() # remove last element
        if self._data:
            self._data[0] = last
            self._sift_down(0)
        return top

    def _sift_down(self, i):
        n = len(self._data)
        while True:
            l, r = self._left(i), self._right(i)
            largest = i
            if l < n and self._greater(l, largest):
                largest = l
            if r < n and self._greater(r, largest):
                largest = r
            if largest == i:
                break
            self._swap(i, largest)
            i = largest
```

```

        self._swap(i, largest)
        i = largest

    def peek_max(self):
        """Return the highest-priority item without removing it."""
        return self._data[0] if self._data else None

    def size(self):
        return len(self._data)

# -----
# Email: stores the parsed fields plus a comparable priority key.
# -----
CATEGORY_RANK = {
    "Boss": 5,
    "Subordinate": 4,
    "Peer": 3,
    "ImportantPerson": 2,
    "OtherPerson": 1,
}

class Email:
    def __init__(self, category, subject, date_str):
        self.category = category
        self.subject = subject
        self.date_str = date_str
        self.rank = CATEGORY_RANK[category]
        self.date_sort_key = self._date_key(date_str)

    @staticmethod
    def _date_key(date_str):
        # date_str is MM-DD-YYYY -> convert to (YYYY, MM, DD) so that
        # tuple comparison sorts chronologically (newer date = larger tuple).
        mm, dd, yyyy = date_str.split("-")
        return (int(yyyy), int(mm), int(dd))

    def priority_key(self):
        # Higher category rank wins; ties broken by newer date.
        return (self.rank, self.date_sort_key)

    def display(self):
        print("Next email:")
        print(f"\tSender: {self.category}")
        print(f"\tSubject: {self.subject}")
        print(f"\tDate: {self.date_str}")

# -----
# Command processing
# -----
def process_file(filepath):
    heap = MaxHeap()

    with open(filepath, "r") as f:
        for raw_line in f:
            line = raw_line.rstrip("\n")
            if not line.strip():
                continue

            if line.startswith("EMAIL "):
                rest = line[len("EMAIL "):]
                category, subject, date_str = rest.split(",")
                heap.push(Email(category, subject, date_str))

            elif line == "COUNT":
                print(f"There are {heap.size()} emails to read.")
                print()

            elif line == "NEXT":
                top = heap.peak_max()
                if top is None:
                    print("There are no emails to read.")
                else:
                    top.display()
                    print()

            elif line == "READ":
                heap.pop_max() # does nothing if heap is empty

            else:
                # Unknown/blank command; ignore silently.
                continue

def main():

```

```
if len(sys.argv) != 2:
    print("Usage: python ceo_email_priority.py <test_file>")
    sys.exit(1)

process_file(sys.argv[1])

if __name__ == "__main__":
    main()
```